Response to the committee

A Lightweight Canonicalization Engine in C for Disciplined Convex Programming

William Zijie Zhang (2362162), Master's thesis, Electrical Engineering, Polytechnique Montréal

Supervisor: Prof. Antoine Lesage-Landry

October 28, 2026

To Prof. Sébastien Le Digabel (chair), Prof. Antoine Legrain (second member) and Prof. Antoine Lesage-Landry (supervisor).

Message au jury

Bonjour chers membres du jury,

Je voulais tout d'abord vous remercier pour vos commentaires sur mon mémoire. Comme vous m'avez demandé de clarifier et d'approfondir plusieurs points des résultats, j'ai pris le temps de les rendre plus complets, et j'ai aussi découvert quelques erreurs dans les affirmations que j'avais faites sur les contributions.

Premièrement, il y a maintenant quatre problèmes qui sont plus lents. J'explique la cause de chacun au chapitre 4 (§4.1), avec la part de chaque phase du temps, et je donne le détail à la section 4. Trois des quatre problèmes peuvent devenir plus rapides. Deux d'entre eux le deviennent avec des changements dans le « frontend » (par frontend, je fais référence à la manipulation des arbres d'expression avant le calcul des dérivées). Par exemple, pouvoir détecter que `D @ ones` (où `D` est une matrice quelconque) équivaut à `D.sum(axis=1)`, la somme sur les colonnes, est un grand avantage. Le troisième, ConvexPlasticity, le devient avec un test de densité. Ces changements ne règlent pas tout : une partie du ralentissement vient de `SparseDiffEngine` lui-même (voir Q5).

Deuxièmement, j'ai refait toutes les mesures sur un ordinateur plus récent (MacBook M2 de 2022, avec Apple `Accelerate`), plutôt que sur l'ordinateur Intel de 2018 de la version soumise, pour que les résultats soient reproductibles sur du matériel actuel, comme celui de la plupart des utilisateurs de `CVXPY`. Pour la première canonicalisation, les ratios et les temps sont semblables. Pour les re-résolutions paramétriques, en revanche, `diffengine` n'est plus rapide que sur trois des sept problèmes, contre cinq dans la version soumise. Sur les quatre autres, la passe numérique ne prend que 4 à 24 % d'une re-résolution ; le reste est du post-traitement, après `SparseDiffEngine`. J'ai donc modifié les affirmations de ma conclusion et de la discussion des résultats pour dire seulement qu'il y a des bénéfices, mais que la séparation ne remplace pas encore le tenseur pour les re-résolutions. En refaisant ces mesures avec les versions des logiciels de la version soumise, j'ai vérifié que ce changement ne vient ni de l'ordinateur ni de `SparseDiffEngine`, mais de l'interface entre `CVXPY` et `SparseDiffEngine`, en particulier d'une modification postérieure à la version 1.9.2. Comme les autres backends sont mesurés avec la version 1.9.2, les re-résolutions de `diffengine` sont maintenant mesurées sans cette modification aussi. Réduire ce post-traitement est maintenant le premier des

travaux futurs.

Troisièmement, le problème UnconstrainedQP n'était pas réellement 100 fois plus rapide. C'était dû à un changement de frontend que j'avais appliqué sur la branche du backend `diffengine`, mais pas sur celles des autres. En l'appliquant à tous, cet avantage disparaît. Ceci a été rectifié : aucun backend n'utilise maintenant ce changement.

À part ces trois points, qui sont les changements majeurs, j'ai aussi fait les études d'ablation et exécuté plus d'instances de benchmark, avec des mesures de signification statistique, ce qui répond à tous vos commentaires.

J'ai rédigé des ébauches de réponses à vos questions, que Claude, l'outil d'IA déclaré dans le mémoire, a ensuite peaufinées ; les chiffres viennent directement des scripts de mesure.

1 Summary of corrections

I have made all the modifications required by the committee and by Prof. Antoine Legrain's examiner report, as well as the minor corrections that Prof. Sébastien Le Digabel annotated and those listed in the appendix of Prof. Legrain's report. The minor corrections are not listed here. All changes since the version sent to the committee are marked in `Document-diff.pdf`, with added text in blue and deleted text in red. Section, table and page numbers below refer to the final version.

Table 1 lists the required modifications. "J" refers to the committee's requirements after the defence, and (i)–(vii) to the items of the synthesis of the examiner report. The defence questions are answered in Section 2; the other items are in the thesis sections listed.

Table 1: Required modifications and where they are made.

Request	Change	Where
J1a: statistical significance	30 runs per comparison; Mann–Whitney [1] per problem, Holm-adjusted [2]; Wilcoxon [3] and a bootstrap interval over the suite.	Ch. 4 introduction, p. 51; §4.1
J1b, (ii): ablation	Dense blocks turned off on all 25 problems, in time and memory; the split ablated on the re-solves (Q2).	§4.1, p. 53; Tables 6 and 7
J1c: more instances	New parameter values for each run, and every problem scaled in size and density.	Ch. 4 introduction; §4.1
J1d: sensitivity to size and sparsity	All 25 problems at five sizes and eight at seven densities, with and without the dense blocks.	§4.1, p. 53; Figure 1
J1e, (iii): non-DPP problems	Re-solves of two families in DPP, non-DPP and parameter-free variants, and of ConvexPlasticity (Q3).	§4.2, p. 56

Request	Change	Where
J2, (i): related literature and DNLP	Matrix-free modelling, `CVXcanon`, `CVXGEN`, `Convex.jl` 0.16, `Pyomo`, block-sparse antecedents and sparse factorization; who leads `SparseDiffEngine` and what is specific to this work (Q6).	§2.2, §1.5, §3.1.1
J3: declaration of AI use	New front-matter page.	Front matter
(iv): repeated measurements	30 runs, each in a fresh process; tables give mean ± standard deviation.	Ch. 4 introduction; Table 4.1
(v): dimensions of the instances	Dimensions, nonzeros, density, parameter size and DPP status of every problem.	Appendix A, Tables A.1 and A.2
(vi): time by phase	The share of each phase on the four slower problems, with the cause of each.	§4.1, p. 52; Table 8
(vii): correctness check	The problem data are checked against the `CPP` backend to a relative tolerance of 10^{-9}, at the first canonicalization and after a parameter update.	Ch. 4 introduction

2 Replies to the defence questions

Q1. *Why materialize dense blocks rather than leave them as operators, as in the matrix-free approach of Diamond and Boyd [4]? When is each approach preferable?*

Because the solvers that `CVXPY` calls need the matrices. Its default solvers are `Clarabel`, an interior-point method, and `OSQP` and `SCS`, which are first-order methods; commercial solvers such as `MOSEK` and `Gurobi` are interior-point or simplex methods. All of them factorize a matrix built from $\mathbf{A}$ and $\mathbf{P}$: an interior-point method factorizes a KKT system at every iteration, and `OSQP` and `SCS`, in their default direct mode, factorize their KKT matrix once and reuse the factorization across iterations. So every one of them needs $\mathbf{A}$ and $\mathbf{P}$ explicitly, whatever its algorithm. Matrix-free modelling pays off for very large problems built from fast transforms, solved by first-order methods that only need products with $\mathbf{A}$ and $\mathbf{A}^\top$, such as the indirect mode of `SCS`. A permuted dense block lies between the two: it is a materialized matrix, but it keeps the structure of the operator, namely its row and column selectors and its dense block.

Changes in the thesis, §2.2, p. 25

In convex optimization, Diamond and Boyd take the matrix-free view. Instead of forming $\mathbf{A}$, they represent the problem as a graph of linear operators, available only through their forward and adjoint products, which suits first-order solvers. The solvers that `CVXPY` calls by default instead factorize a matrix built from $\mathbf{A}$ and $\mathbf{P}$. The matrix-free approach pays off on very large problems built from fast transforms, such as convolutions and Fourier transforms, and it needs solvers that accept operators instead of matrices. The permuted dense matrices of §3.2

sit between the two: they form the matrix that the solver factorizes, but keep the structure of the operator, its row and column selectors and its dense block.

Q2. *If the permuted dense representations are disabled, what fraction of the speedup remains?*

Most of it. Without the dense blocks, `diffengine` is 2.15× faster than the best `CVXPY` backend in geometric mean, against 2.29× with them, and the difference is not significant over the suite (Wilcoxon, $p = 0.07$). Per problem, however, the effect is large in both directions (Table 4.1 of the thesis; Tables 6 and 7): the dense blocks help by 1.8 to 3× on eight problems with dense data, and hurt by 1.4 to 46× on the three that build a dense block from sparse data (CVaR, ConvexPlasticity, UnconstrainedQP). Without them, `diffengine` also uses 0.83 times the peak memory in geometric mean.

The dense blocks are the only part of the engine that can be turned off on a first canonicalization; the implementation in C, one Jacobian per node and the global layout are built into every atom, so we do not attribute the remaining 2.15× to any one of them. The split between the symbolic and numeric passes matters only on re-solves, where removing it makes a re-solve 2.9× slower, significantly on all seven problems (Table 2). *We conclude that most of the first-canonicalization speedup comes from the engine as a whole rather than from the dense blocks, which should only be built from data that is dense in fact, and that the re-solve speed comes from the split.*

Table 2: The ablations in summary: geometric mean of the ratio of each variant to its reference, its bootstrap 95 % confidence interval, the Wilcoxon p-value over the problems, and the number of problems with a significant difference (Mann–Whitney, Holm-adjusted).

Variant	Reference	Ratio	95 % CI	p	Significant
First canonicalization, 25 *problems*					
no dense blocks	`diffengine`	1.06	0.71–1.45	0.067	24/25
no dense blocks	best `CVXPY`	0.46	0.36–0.61	3.2×10^{-5}	25/25
Re-solve, 7 *DPP problems*					
no dense blocks	`diffengine`	1.01	0.98–1.05	0.938	6/7
no split	`diffengine`	2.86	1.73–4.91	0.016	7/7
neither	`diffengine`	3.68	2.19–6.20	0.016	7/7

Q3. *How does re-solving behave on a non-DPP parametric problem?*

On the two families written with a product of two parameters, which DPP forbids, `CVXPY` has to canonicalize the problem again at every solve, while `diffengine` keeps its symbolic pass and reruns only the numeric pass. It re-solves as fast as on the DPP variant, 13 to 14× faster than `CVXPY` on `lasso` and `factor_cov`, significantly on both (Table 3). ConvexPlasticity, the suite's only natural non-DPP parametric problem, is the exception: it re-solves in 207 ms against 42 ms for `CVXPY`, because 88 % of the re-solve assembles a dense 3000 × 3000 block of $\mathbf{P}$ built from a diagonal. Without the dense blocks, it re-solves in 2.0 ms, 21× faster than `CVXPY`. The thesis no longer claims that the split replaces the tensor (Q4).

Table 3: Re-solve time in milliseconds, mean over 30 runs; every standard deviation is below 1 % of the mean. `factor_cov` is FactorCovarianceModel with n assets. The non-DPP variant writes λ, or the risk aversion γ, as the product of two parameters, and the parameter-free variant makes it a constant. `CVXPY` applies the tensor on the DPP variant and canonicalizes again on the others; the cached column of `diffengine` runs the numeric pass only, and the rebuild column the symbolic pass as well. The ratio is the median of `diffengine`, cached when it can, over that of `CVXPY`, * when significant. The thesis states these results in §4.2 without the table.

			`diffengine`		
Family	Variant	`CVXPY`	cached	rebuild	Ratio
`lasso`, $n = 1000$	DPP	6.4	23.3	32.7	3.64*
	non-DPP	315	23.3	32.8	0.07*
	parameter-free	338	—	49.0	0.14*
`factor_cov`, $n = 5000$	DPP	1.2	3.6	6.9	2.86*
	non-DPP	45.8	3.6	7.0	0.08*
	parameter-free	47.1	—	11.0	0.23*

Changes in the thesis, §4.2, p. 56

On the non-DPP variants, `CVXPY` canonicalizes again at every solve, while `diffengine` reruns only its numeric pass and re-solves 13 to 14 times faster. ConvexPlasticity, the only parametric problem of the suite that is not DPP, is the exception [...]. Without the dense blocks, it re-solves in 2.0 milliseconds, 21 times faster than `CVXPY`.

Changes in the thesis, §4.3, p. 57

Caching canonicalization does not necessarily require the DPP ruleset. [...] the symbolic and numeric split re-solves within a factor of four of the cached tensor on every DPP problem. [...] it re-solves problems that are not DPP, on which `CVXPY` has to canonicalize again, 13 to 14 times faster.

Q4. *Why not re-evaluate only the subtrees that depend on the updated parameters?*

The engine now does this: after a parameter update, the numeric pass re-evaluates only the subtrees that contain a parameter, and keeps the cached values of the others. All re-solve times in the thesis use it.

On the seven DPP problems (Table 4), `diffengine` has the fastest first canonicalization on all seven, 2.5 to 6.6× faster than the fastest tensor build. The cached tensor re-solves faster on four of them, by 1.10 to 3.60×, and recovers its slower first canonicalization only after 18 to 124 re-solves. On those four, the numeric pass takes only 4 to 24 % of a re-solve; the rest is post-processing after the engine, the conversion to the solver's format. *We conclude that the split does not replace the tensor on DPP problems that are re-solved many times, but comes within a factor of four of it without the DPP rules.* Making the post-processing cheaper is now the first item of future work.

Table 4: Table 4.2 of the thesis (p. 56): first canonicalization in seconds and re-solve in milliseconds, mean over 30 runs. A dash is a tensor that exceeded the memory or time limit. The ratio is the median re-solve of `diffengine` over that of the fastest tensor, * when significant.

	first canonicalization (s)				re-solve (ms)		
Benchmark	`CPP`	`SCIPY`	`COO`	`diffengine`	tensor	`diffengine`	Ratio
ParamSmallMatrixStuffing	2.02	7.30	0.083	**0.033**	1.8	**0.4**	0.24*
ParamConeMatrixStuffing	0.74	1.85	0.055	**0.022**	1.0	**0.3**	0.27*
SimpleFullyParametrizedLP	—	—	0.57	**0.12**	47	**35**	0.73*
SimpleScalarParametrizedLP	1.33	1.17	1.19	**0.23**	**63**	70	1.10*
ParametrizedQP	—	—	2.63	**0.76**	**299**	349	1.16*
SVMWithL1Regularization	2.80	2.57	2.83	**0.39**	**38**	125	3.26*
FactorCovarianceModel	1.20	1.14	1.19	**0.18**	**21**	74	3.60*

Changes in the thesis, §3.4.1, p. 47

The numeric pass therefore skips an affine tree after the first evaluation, until the next parameter update. After an update, it re-evaluates only the subtrees that contain a parameter, and keeps the cached values of the others.

Q5. *When does the global layout become a handicap relative to `CVXPY`'s local layout, in particular for memory?*

When the problem has many variables and many nodes. In the global layout, each node keeps, besides its Jacobian, integer work arrays of length n, the total number of variables, whatever variables its own subtree contains. Their memory is therefore $O(\text{nodes} \times n)$ integers. In `CVXPY`'s local layout, each node stores blocks only for the variables it depends on, so the same overhead is proportional to the sum, over the nodes, of the number of variables each one depends on. The two coincide when every node depends on every variable, and the local layout wins when most nodes depend on a few variables out of many.

OptimalAdvertising shows it. Each of its 250 minimum terms depends on one row of a 250×1000 variable, but has a Jacobian over all 250 000 variables. Without the dense blocks, its peak memory is 945 MB, against 173 to 197 MB for the `CVXPY` backends. With them it reaches 6.1 GB, because each dense block of the aggregate constraints $\mathbf{D}^\top \mathbf{1} \leq \mathbf{t}$ and $\mathbf{D1} \geq \mathbf{c}$ also keeps one integer per variable of the problem: the same cost, paid per block rather than per node. Rewriting these products with a vector of ones as sums avoids the dense blocks: OptimalAdvertising then canonicalizes in 0.33 s instead of 0.91 s, faster than the 0.53 s of `CVXPY`, with 1.4 GB instead of 6.1 GB. Storing the per-block maps sparsely would fix the general case.

Q6. *What part of `SparseDiffEngine` pre-existed the thesis or was developed for DNLP, and what part is specific to this work?*

Daniel Cederberg leads `SparseDiffEngine`. He started it as the differentiation engine of the `DNLP` extension of `CVXPY` [5], which differentiates nonlinear atoms at every iterate of the

solver, and I co-developed it with him. The version sent to the committee should have said so clearly; it now does. This thesis uses the engine for a different purpose, canonicalization: it evaluates the affine atoms and the quadratic form at the origin to obtain the problem data. What is specific to the thesis is the interface to `CVXPY`, namely the `diffengine` backend and its re-solve path for parametric problems, and the benchmarks of Chapter 4.

Changes in the thesis, §1.5, p. 16

Daniel Cederberg is the lead developer of `SparseDiffEngine`. It was first built as the differentiation engine of the `DNLP` extension of `CVXPY`, where it differentiates nonlinear atoms such as `exp`, `log` and `entr` at every iterate of the solver. This thesis builds on the observation that canonicalization is a special case of the same computation: once a problem is canonicalized, only affine atoms and quadratic forms remain, and their derivatives are the problem data.

Changes in the thesis, §3.1.1, p. 37

What is specific to this thesis is the interface to `CVXPY` of §3.3, namely the `diffengine` backend and its re-solve path for parametric problems, and the benchmarks of Chapter 4.

Q7. *What is the strategy for BLAS and binary distribution of* ***`sparsediffpy`****?*

`cvxcore` is a C++ extension that lives inside the `CVXPY` repository, so `CVXPY` has to compile and ship it. `sparsediffpy` is a separate package that `CVXPY` only depends on. Its compiled wheels for Linux, macOS and Windows are published on `PyPI`; they link Apple's `Accelerate` on macOS and bundle `OpenBLAS` elsewhere, so users install nothing beyond the package.

3 New and revised tables

The thesis adds the two tables of Appendix A and redoes two existing tables with the new measurements. The other results are stated in the text, with the full measurements in Section 4.

Table 5: Tables added to the thesis, or revised.

Item	Content	Request
Table 4.1	Revised: first canonicalization of the 25 problems, now mean ± standard deviation over 30 runs with significance, and the ratio without the dense blocks, p. 55	J1a, J1b, (ii), (iv)
Table 4.2	Revised: first canonicalization and re-solve of the DPP problems, with standard deviations and significance, p. 56	J1a, (iv), Q4
Table A.1	The 25 benchmark problems, p. 71	(v)
Table A.2	Dimensions of the benchmark problems, p. 73	(v)

4 Supplementary results

The thesis states the results below in a few sentences each; the full measurements are given here for the committee. All are measured as in Chapter 4, in 30 runs.

Dense blocks: per problem (J1b, (ii), Q2)

Table 6: First canonicalization of `diffengine` with and without the dense blocks, in milliseconds. The ratio is the median time without the dense blocks over that with them, * when significant (Mann–Whitney, Holm-adjusted $p < 0.05$). The last two columns are the ratio of `diffengine` to the best `CVXPY` backend, as in Table 4.1 of the thesis.

				to best `CVXPY`	
Benchmark	`diffengine`	no dense blocks	Ratio	with	without
Dense blocks help					
SVMWithL1Regularization	387 ± 1	745 ± 5	1.92*	0.15	0.28
FactorCovarianceModel	177 ± 3	366 ± 6	2.06*	0.15	0.32
Yitzhaki	145 ± 1	263 ± 1	1.82*	0.15	0.28
SimpleQP	453 ± 1	914 ± 6	2.01*	0.16	0.32
HuberRegression	495 ± 8	1 015 ± 3	2.06*	0.18	0.37
LeastSquares	435 ± 2	906 ± 2	2.08*	0.18	0.38
SimpleLP	1 121 ± 3	1 485 ± 5	1.32*	0.19	0.25
ConeMatrixStuffing	642 ± 4	1 941 ± 7	3.03*	0.30	0.90
SmallMatrixStuffing	502 ± 2	1 369 ± 6	2.73*	0.30	0.81
ParamSmallMatrixStuffing	32.7 ± 0.1	40.4 ± 0.3	1.23*	0.52	0.64
ParamConeMatrixStuffing	22.4 ± 0.4	27.4 ± 0.7	1.22*	0.52	0.64
SDPSegfault1132	2 147 ± 6	3 466 ± 15	1.61*	1.32	2.12
Little effect					
SimpleScalarParametrizedLP	230 ± 1	225 ± 0.3	0.98*	0.21	0.21
SimpleFullyParametrizedLP	115 ± 0.5	115 ± 0.4	1.00*	0.22	0.22
SlowPruning	408 ± 1	417 ± 1	1.02*	0.23	0.23
ParametrizedQP	755 ± 46	746 ± 34	1.00	0.25	0.25
Cajas	205 ± 1	209 ± 1	1.02*	0.30	0.30
TvInpainting	257 ± 1	250 ± 1	0.97*	0.32	0.31
SemidefiniteProgramming	405 ± 3	406 ± 1	1.00*	0.64	0.64
QuantumHilbertMatrix	653 ± 3	671 ± 3	1.03*	0.83	0.85
Murray	1 181 ± 1	1 179 ± 2	1.00*	0.96	0.96
OptimalAdvertising	901 ± 4	834 ± 1	0.93*	1.68	1.56
Dense blocks hurt					
CVaR	3 421 ± 9	2 419 ± 36	0.71*	0.53	0.37
ConvexPlasticity	507 ± 1	97.1 ± 3.2	0.19*	7.31	1.39

Benchmark	`diffengine`	no dense blocks	Ratio	with	without
UnconstrainedQP	22 937 ± 21	500 ± 2	0.02*	11.73	0.26
Geometric mean			**1.06**	**0.44**	**0.46**

Table 7: Growth of the peak resident memory during the first canonicalization, in megabytes. The last column is `diffengine` without the dense blocks.

Benchmark	`CPP`	`SCIPY`	`COO`	`diffengine`	no dense blocks
SVMWithL1Regularization	1 976	2 041	2 904	2 034	2 417
FactorCovarianceModel	1 129	1 047	1 334	760	999
Yitzhaki	449	437	504	476	572
SimpleQP	2 194	2 355	3 236	1 655	2 204
HuberRegression	2 223	2 246	3 121	1 413	1 866
LeastSquares	2 142	2 327	3 021	1 459	1 893
SimpleLP	5 353	4 961	5 027	4 792	5 057
SimpleScalarParametrizedLP	1 088	1 024	1 068	1 009	1 009
SimpleFullyParametrizedLP	517	494	526	492	492
SlowPruning	1 352	1 425	1 828	1 797	1 120
ParametrizedQP	2 035	2 174	2 998	1 775	1 775
Cajas	1 193	298	320	341	342
ConeMatrixStuffing	2 010	964	3 730	478	1 623
SmallMatrixStuffing	1 189	634	2 405	322	1 117
TvInpainting	705	858	994	1 048	1 048
ParamSmallMatrixStuffing	17.4	12.4	21.9	10.6	10.5
ParamConeMatrixStuffing	9.8	8.2	12.2	5.9	5.8
CVaR	4 022	2 804	3 587	10 654	2 115
SemidefiniteProgramming	1 317	350	640	1 138	1 138
QuantumHilbertMatrix	222	218	288	1 194	1 194
Murray	2 849	220	305	306	307
SDPSegfault1132	7 536	3 460	15 628	10 413	12 885
OptimalAdvertising	173	182	197	6 066	945
ConvexPlasticity	82.5	82.5	82.3	1 329	25.5
UnconstrainedQP	1 427	1 058	2 105	1 754	644

Density sweep (J1d)

The data matrix of eight problems is multiplied entrywise by a random 0/1 mask with a fraction ρ of ones, and stays a dense array, as it would be in a user's code. Figure 1 shows four of them; FactorCovarianceModel, SimpleQP, Yitzhaki and `lasso` follow the pattern of LeastSquares.

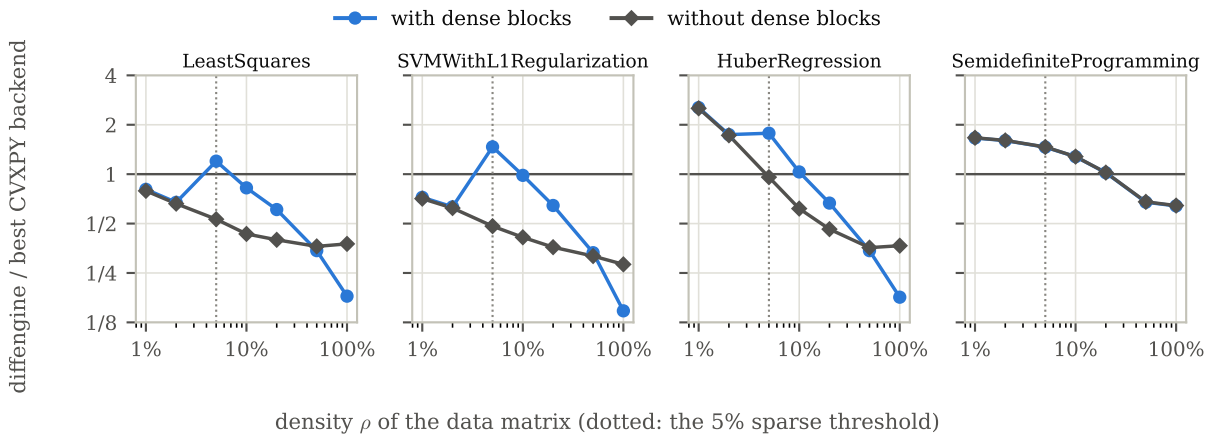


Figure 1: Ratio of the median canonicalization time of `diffengine` to that of the best `CVXPY` backend as the density ρ of the data matrix falls, with and without the dense blocks. The dotted line is the 5 % density below which `diffengine` stores a constant factor in the sparse format. Every difference is significant (Mann–Whitney, Holm-adjusted over the problems of each density).

The four slower problems ((vi), Q5)

On the four problems on which `diffengine` is slower, the cause lies in how their data reaches the engine. Table 8 gives the share of each phase of its first canonicalization. Three of the four are addressed by changes to `CVXPY`, described below, which the measurements of the thesis do not include; each fix is measured alone.

Table 8: Phases of the first canonicalization of `diffengine` on the four slower problems, in percent of its time, and the median share over the 25 problems.

Benchmark	Time (s)	Front end	Convert	Symbolic	Numeric	Assemble	Format
UnconstrainedQP	22.94	0	0	2	98	0	0
ConvexPlasticity	0.51	6	2	47	6	38	0
OptimalAdvertising	0.90	4	1	89	4	1	1
SDPSegfault1132	2.15	0	0	49	47	3	0
Median, all 25	—	4	9	19	14	12	12

UnconstrainedQP. The problem minimizes $\|\mathbf{F}^{\mathsf{H}}\mathbf{E}(\mathbf{z})\mathbf{F}\mathbf{x}-\mathbf{y}\|_2$, with $\mathbf{E}(\mathbf{z}) = \mathbf{I}_{14}\otimes\operatorname{diag}(\mathbf{z})$, over $\mathbf{z}\in\mathbb{C}^{18}$, where $\mathbf{F}$ is the 252×252 discrete Fourier transform matrix. Evaluated from the left, the product $(\mathbf{F}^{\mathsf{H}}\mathbf{E})\mathbf{F}$ has a dense Jacobian of $63\,504\times36$ entries, which the numeric pass fills one entry at a time. Evaluated from $\mathbf{x}$, as the reassociation of matrix products excluded from the thesis measurements does, every intermediate is a vector, and the canonicalization takes 16 milliseconds instead of 22.9 seconds, with a peak memory of 53 megabytes instead of 1.75 gigabytes.

OptimalAdvertising. The problem maximizes $\sum_{i=1}^{250} \min(r_i \mathbf{p}_i^\top \mathbf{d}_i, b_i)$ subject to $\mathbf{D} \geq \mathbf{0}$, $\mathbf{D}^\top \mathbf{1} \leq \mathbf{t}$ and $\mathbf{D}\mathbf{1} \geq \mathbf{c}$, over $\mathbf{D} \in \mathbb{R}^{250 \times 1000}$ with rows $\mathbf{d}_i^\top$. Each minimum term depends on one row of $\mathbf{D}$, but its Jacobian spans all 250 000 variables, and the symbolic pass visits every column of it, so its cost grows with the number of terms times the number of variables. The two aggregate constraints are written as products with a vector of ones, which become dense blocks. A change to `CVXPY` lowers them to sums, which reduces the canonicalization from 0.91 to 0.33 seconds, below the 0.53 seconds of the fastest `CVXPY` backend, and the peak memory from 6.1 to 1.4 gigabytes.

SDPSegfault1132. Three dense blocks of 49.5 million entries cancel in a sum with 1.01 million nonzeros, and `diffengine` carries the cancelled entries through both passes.

ConvexPlasticity. The diagonal $h\mathbf{I}$ of the quadratic objective is given as a dense array and becomes a dense 3000×3000 block of $\mathbf{P}$. A change to `CVXPY` stores $\mathbf{P}$ sparse below 5 % nonzeros, as is already done for a constant factor of a matrix product, which reduces the canonicalization from 509 to 203 milliseconds and the peak memory from 1.33 to 0.59 gigabytes. The problem remains 2.9 times slower than on the fastest `CVXPY` backend, and most of the remaining time is in the symbolic pass.

Re-solve against a full canonicalization (Q4)

Table 9: Re-solve time in milliseconds on the seven DPP problems: the cached tensor of `CVXPY`, a full canonicalization on the `ignore_dpp` path, both with the fastest backend, and the cached re-solve of `diffengine`.

	`CVXPY`		`diffengine`
Benchmark	tensor	recompile	cached
ParamSmallMatrixStuffing	1.84 ± 0.01	57.6 ± 0.2	0.438 ± 0.007
ParamConeMatrixStuffing	0.958 ± 0.005	37.7 ± 0.2	0.265 ± 0.004
SimpleFullyParametrizedLP	46.7 ± 0.1	519 ± 1	34.6 ± 0.8
SimpleScalarParametrizedLP	62.8 ± 0.3	1 078 ± 2	70.4 ± 5.9
ParametrizedQP	299 ± 8	2 868 ± 14	349 ± 16
SVMWithL1Regularization	38.3 ± 0.1	2 570 ± 3	125 ± 2
FactorCovarianceModel	20.5 ± 0.1	1 138 ± 3	74.2 ± 0.8

References

[1] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.

[2] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[3] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[4] S. Diamond and S. Boyd, "Matrix-free convex optimization modeling," in *Optimization and Its Applications in Control and Data Sciences*, ser. Springer Optimization and Its Applications, B. Goldengorin, Ed. Springer, 2016, vol. 115, pp. 221–264.

[5] D. Cederberg, W. Zhang, P. Nobel, and S. Boyd, "Disciplined nonlinear programming," arXiv:2606.02896, 2026. [Online]. Available: https://arxiv.org/abs/2606.02896